Session 27 — Teamwork + Homework Solution Walkthrough

This document explains the official solutions to both the Session 27 TEAMWORK task and the Session 27 HOMEWORK, in simple, step-by-step language. It follows the exact order the official solution was actually built in, and it keeps every file path, every line of code, and every explanation a student needs to fully reproduce both solutions — nothing here is shortened or summarized away. If you only had this one document (plus the TEAMWORK and HOMEWORK task sheets), you should still be able to build the complete, correct solution from start to finish.

It has two parts:

- **PART 1 — Solving the Teamwork task** (the per-project Activity feed) — steps TW-01 to TW-04.
- **PART 2 — Solving the Homework** (Notifications, comment deletion, the Dashboard, navigation, the shared error boundary, and the `@mention` design challenge) — steps HW-01 to HW-22.

A quick reminder before you start: CORE, STRETCH, and CHALLENGE are not "required vs. optional" labels. All three are required parts of this homework. The names only describe how much harder each part is — STRETCH is simply more challenging than CORE, and CHALLENGE asks you to reason and write instead of code.

PART 1 — Solving the Teamwork Task

The goal of TEAMWORK: show a project's own recent activity — who did what, and when — right on the Project Detail page. By the time TEAMWORK starts, LIVE has already given the app real Comments and a real `@mention` autocomplete; TEAMWORK's job is to add the Activity feed on top of that.

Step TW-01 — An `ActivityEntry` type

Goal: Give TypeScript a shape for what one activity entry looks like, matching exactly what the backend's activity endpoint already returns.

Technical context: This is the same habit LIVE already used for `Comment` — add a shared type *before* writing any component that needs it, so every later step can rely on it compiling correctly.

File: `web/src/lib/types.ts`

Exact location: Add the new `ActivityEntry` type right after the existing `Comment` type.

Before:

TSX

```tsx
export type Comment = {
  id: number;
  issueId: number;
  authorId: number | null;
  body: string;
  createdAt: string;
  author: { id: number; name: string } | null;
};
```

Implementation:

TSX

```
export type ActivityEntry = {
  id: number;
  workspaceId: number;
  issueId: number | null;
  actorId: number | null;
  action: string;
  metadata: Record<string, unknown> | null;
  createdAt: string;
  actor: { id: number; name: string } | null;
};
```

After:

TSX

```
export type Comment = {
  id: number;
  issueId: number;
  authorId: number | null;
  body: string;
  createdAt: string;
  author: { id: number; name: string } | null;
};

export type ActivityEntry = {
  id: number;
  workspaceId: number;
  issueId: number | null;
  actorId: number | null;
  action: string;
  metadata: Record<string, unknown> | null;
  createdAt: string;
  actor: { id: number; name: string } | null;
};
```

Why this works: `action` is typed as a plain `string`, not as a TypeScript union of every possible value (`"ISSUE_CREATED" | "STATUS_CHANGED" | ...`). That is the same choice `Issue.status` and `Sprint.status` already made, because the backend is the one source of truth for which action values exist — if the backend adds a new action value tomorrow, the frontend type should not need to change just to keep compiling.

In plain terms: this step is just a shape, like a template. It tells TypeScript "an activity row looks like this" so every later step that touches activity data gets autocomplete and type errors instead of guessing field names.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit, no errors. Nothing uses this new type yet — that is expected.

Common mistakes: None specific to this step — it is a pure type addition.

Step TW-02 — `ActivityFeed`, the basic skeleton

Goal: Create a component that lists every activity entry it is given — for now, just a plain "actor did action" line, with no friendly sentence yet.

Technical context: This reuses the exact list shape `CommentList` already uses: map over an array, and show a friendly empty state when the array is empty.

File: `web/src/components/activity/ActivityFeed.tsx`

Exact location: A brand-new file, in a new `components/activity/` folder.

Before: File does not exist yet.

Implementation:

TSX

```tsx
import type { ActivityEntry } from "@/lib/types";

export function ActivityFeed({ entries }: { entries: ActivityEntry[] }) {
  if (entries.length === 0) {
    return <p className="text-sm text-gray-400">No activity yet.</p>;
  }

  return (
    <ul className="flex flex-col gap-2 text-sm">
      {entries.map((e) => (
        <li key={e.id} className="flex items-center justify-between gap-2 text-gray-500
dark:text-gray-400">
          <span>{e.actor?.name ?? "Someone"} -- {e.action}</span>
          <time className="flex-shrink-0 text-xs text-gray-400">
            {new Date(e.createdAt).toLocaleDateString()}
          </time>
        </li>
      ))}
    </ul>
  );
}
```

After:

TSX

```tsx
import type { ActivityEntry } from "@/lib/types";

export function ActivityFeed({ entries }: { entries: ActivityEntry[] }) {
  if (entries.length === 0) {
    return <p className="text-sm text-gray-400">No activity yet.</p>;
```

```
  }

  return (
    <ul className="flex flex-col gap-2 text-sm">
      {entries.map((e) => (
        <li key={e.id} className="flex items-center justify-between gap-2 text-gray-500
dark:text-gray-400">
          <span>{e.actor?.name ?? "Someone"} -- {e.action}</span>
          <time className="flex-shrink-0 text-xs text-gray-400">
            {new Date(e.createdAt).toLocaleDateString()}
          </time>
        </li>
      ))}
    </ul>
  );
}
```

Why this works: `e.actor?.name ?? "Someone"` matters because the `actor` relation is genuinely nullable on the backend (the foreign key uses `onDelete: SetNull`). If the user who did something is later removed from the system, the activity entry about them should still render correctly, with a harmless fallback label, instead of crashing.

In plain terms: this component doesn't fetch anything and doesn't know where its data comes from — you just hand it a list of entries, and it draws one line per entry. That's what "render-only" means: no network calls, no state, just turning data into UI.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. The friendly, human-readable message comes next (TW-03); nothing renders this component on a real page yet (that's TW-04).

Common mistakes: None specific to this step.

Step TW-03 — `ActivityFeed`, the human-readable message

Goal: Make each entry read as a real sentence (`"Omar created this issue"`) instead of the raw `action` string (`"ISSUE_CREATED"`).

Technical context: This is a pure, mechanical continuation of TW-02 — a lookup table from `action` to a message, the same shape any `switch` over a small, fixed set of string values always takes.

File: `web/src/components/activity/ActivityFeed.tsx`

Exact location: Add a `describe` function before the component, and replace the `<span>`'s content with a call to it.

Before:

TSX

```
export function ActivityFeed({ entries }: { entries: ActivityEntry[] }) {
```

TSX

```
          <span>{e.actor?.name ?? "Someone"} -- {e.action}</span>
```

Implementation:

TSX

```
function describe(entry: ActivityEntry) {
  const actor = entry.actor?.name ?? "Someone";
  switch (entry.action) {
    case "ISSUE_CREATED":
      return `${actor} created this issue`;
    case "STATUS_CHANGED":
      return `${actor} changed the status`;
    case "ASSIGNED":
      return `${actor} changed the assignee`;
    case "COMMENTED":
      return `${actor} commented`;
    default:
      return `${actor} updated this issue`;
  }
}

export function ActivityFeed({ entries }: { entries: ActivityEntry[] }) {
```

TSX

```
          <span>{describe(e)}</span>
```

After:

TSX

```
function describe(entry: ActivityEntry) {
  const actor = entry.actor?.name ?? "Someone";
  switch (entry.action) {
    case "ISSUE_CREATED":
      return `${actor} created this issue`;
    case "STATUS_CHANGED":
      return `${actor} changed the status`;
    case "ASSIGNED":
      return `${actor} changed the assignee`;
    case "COMMENTED":
      return `${actor} commented`;
    default:
      return `${actor} updated this issue`;
  }
}
```

```
export function ActivityFeed({ entries }: { entries: ActivityEntry[] }) {
  if (entries.length === 0) {
    return <p className="text-sm text-gray-400">No activity yet.</p>;
  }

  return (
    <ul className="flex flex-col gap-2 text-sm">
      {entries.map((e) => (
        <li key={e.id} className="flex items-center justify-between gap-2 text-gray-500
dark:text-gray-400">
          <span>{describe(e)}</span>
          <time className="flex-shrink-0 text-xs text-gray-400">
            {new Date(e.createdAt).toLocaleDateString()}
          </time>
        </li>
      ))}
    </ul>
  );
}
```

Why this works: The `default` branch is the important detail. This is deliberately NOT an exhaustive `switch` over every possible action value. `MENTIONED`, `SPRINT_STARTED`, and any other action the backend might add later all fall through to one generic message ("updated this issue"). That means a future backend change can never break this component — it just shows a slightly less specific sentence for an action it doesn't recognize yet, instead of crashing or showing nothing.

In plain terms: think of `describe()` as a translator. The backend speaks in short codes (`"STATUS_CHANGED"`); this function turns each code into a normal English sentence. The `default` case is a safety net — "if I don't recognize this code, still say *something* sensible."

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. Nothing renders this component on a real page yet (TW-04).

Common mistakes: None specific to this step.

Step TW-04 — Wire `ActivityFeed` into Project Detail (TEAMWORK complete)

Goal: Project Detail now actually shows this project's own recent activity.

Technical context — read this one carefully, it's the most important step in TEAMWORK. The backend's activity endpoint is workspace-wide, not project-scoped — there is no `GET /projects/:id/activity` route. This is the same situation Backlog (Session 26) and `SortableIssueList` (Session 24) already faced: the backend sometimes returns more data than one page actually needs, and it is the frontend's job to narrow it down. The subtle part is WHICH set of issues you use to do that narrowing.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/page.tsx`

Exact location: Two new imports; the existing issues fetch grows a second, unfiltered fetch plus the activity fetch and filter; the returned JSX grows an Activity section.

Before:

TSX

```
import type { Issue, Project } from "@/lib/types";
```

TSX

```
import { SortableIssueList } from "@/components/issues/SortableIssueList";
```

TSX

```
  const issuesRes = await apiFetch(`/issues?
projectId=${project.id}${statusQuery}${titleQuery}`);
  const issues: Issue[] = await issuesRes.json();
```

TSX

```
      <SortableIssueList issues={issues} basePath={`/workspaces/${id}/projects/${project.id}`}
/>
    </main>
```

Implementation:

TSX

```
import type { Issue, Project, ActivityEntry } from "@/lib/types";
```

TSX

```
import { SortableIssueList } from "@/components/issues/SortableIssueList";
import { requireSessionToken } from "@/lib/session";
import { ActivityFeed } from "@/components/activity/ActivityFeed";
```

TSX

```
  const token = await requireSessionToken();
  const issuesRes = await apiFetch(`/issues?
projectId=${project.id}${statusQuery}${titleQuery}`);
  const issues: Issue[] = await issuesRes.json();

  // A second, UNFILTERED fetch -- this project's full issue set, never narrowed by whatever
  // status/title filter the Issues UI above currently has applied.
  const allIssuesRes = await apiFetch(`/issues?projectId=${project.id}`);
  const allIssues: Issue[] = await allIssuesRes.json();

  const activityRes = await apiFetch(`/workspaces/${project.workspaceId}/activity`, { token });
  const allActivity: ActivityEntry[] = await activityRes.json();
  const issueIds = new Set(allIssues.map((i) => i.id));
  const activity = allActivity.filter((e) => e.issueId !== null && issueIds.has(e.issueId));
```

TSX

```
      <SortableIssueList issues={issues} basePath={`/workspaces/${id}/projects/${project.id}`}
/>

      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Activity
      </h2>
      <ActivityFeed entries={activity} />
    </main>
```

After:

TSX

```
import type { Issue, Project, ActivityEntry } from "@/lib/types";
```

TSX

```
import { SortableIssueList } from "@/components/issues/SortableIssueList";
import { requireSessionToken } from "@/lib/session";
import { ActivityFeed } from "@/components/activity/ActivityFeed";
```

TSX

```
  const token = await requireSessionToken();
  const issuesRes = await apiFetch(`/issues?
projectId=${project.id}${statusQuery}${titleQuery}`);
  const issues: Issue[] = await issuesRes.json();

  // A second, UNFILTERED fetch -- this project's full issue set, never narrowed by whatever
  // status/title filter the Issues UI above currently has applied.
  const allIssuesRes = await apiFetch(`/issues?projectId=${project.id}`);
  const allIssues: Issue[] = await allIssuesRes.json();

  const activityRes = await apiFetch(`/workspaces/${project.workspaceId}/activity`, { token });
  const allActivity: ActivityEntry[] = await activityRes.json();
  const issueIds = new Set(allIssues.map((i) => i.id));
  const activity = allActivity.filter((e) => e.issueId !== null && issueIds.has(e.issueId));
```

TSX

```
      <SortableIssueList issues={issues} basePath={`/workspaces/${id}/projects/${project.id}`}
/>

      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Activity
      </h2>
      <ActivityFeed entries={activity} />
    </main>
```

Why this works — the full explanation:

There are now TWO different issue-related lists on this page, and they serve two completely different jobs:

- `issues` — fetched WITH `statusQuery` / `titleQuery`, i.e. filtered by whatever the Issues UI's search box or status filter currently says. This is the list that drives `SortableIssueList` (what the student actually sees in the board/list view). It never changes.
- `allIssues` — fetched WITHOUT any filter, just `?projectId=${project.id}`. This is the list used ONLY to decide which activity entries belong to this project. It is brand new in this step.

Why do we need a second, separate fetch instead of reusing `issues`? Because `issueId` identifies an ISSUE, and `project.id` identifies a PROJECT — two unrelated kinds of id that happen to both be plain numbers, but comparing them directly (`entry.issueId === project.id`) would be a bug, not a shortcut. The only correct way to know "does this activity entry belong to this project" is to check whether its `issueId` is one of this project's own issue ids — which means you first need the full list of this project's issue ids.

Here's the part that is easy to get wrong: if you build that id-set from the FILTERED `issues` list instead of a separate unfiltered fetch, the Activity feed would quietly break the moment someone uses the Issues filter. Concrete example: a project has Issue A and Issue B, and both have real activity (a comment was posted on each). If a user filters the Issues UI down to "Open only" and Issue B happens to be closed, Issue B disappears from `issues`. If Activity membership were computed from that same filtered `issues` list, Issue B's activity would ALSO disappear from the Activity section — even though Issue B still very much belongs to this project. That is wrong. The correct behavior is: Activity keeps showing entries for BOTH Issue A and Issue B, no matter what the Issues filter is currently set to, because Activity membership and the visible Issues list are two independent questions. That is exactly why `allIssues` is fetched separately, with no filter applied at all, and why `issueIds` / `activity` are built from `allIssues`, never from `issues`.

One more detail: `e.issueId !== null && issueIds.has(e.issueId)`. Some activity entries are not about any specific issue at all (for example, a future "member added to workspace" action would have `issueId: null`). The `e.issueId !== null` check excludes those from a per-project view the same way it excludes an entry about a completely different project's issue — both cases simply don't belong here.

In plain terms: `issues` answers "what should the Issues list on screen show right now, given the current filter?" `allIssues` answers a totally different question: "what are ALL of this project's issues, regardless of any filter?" Activity membership must always use the second answer. If you ever find yourself reusing the filtered list for Activity, that's the bug this step exists to prevent.

Verify / test:

BASH

```
npx tsc --noEmit && npm run build
```

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1 | grep -o
"Activity"
```

Also verify by hand: open a project containing at least two issues with existing activity (e.g. a comment on each). Apply a status/title filter in the Issues UI that hides one of them from `SortableIssueList`. Confirm the Activity section still lists BOTH issues' entries — filtering the Issues list must never remove an issue's own activity from this project's feed. Confirm a DIFFERENT project's activity stays excluded throughout.

Expected result: `tsc` / `build` both exit clean; the Activity section appears, listing this project's own recent entries, completely unaffected by the Issues UI's current filter. TEAMWORK is complete: a real `ActivityEntry` type, a real `ActivityFeed` component, and a real per-project filtered render, reachable directly from Project Detail.

Common mistakes:

- Fetching activity without `{ token }` — this route requires authentication; a missing token produces a `401`, not an empty list.
- Building the Activity id-set from the FILTERED `issues` list instead of the separate, always-unfiltered `allIssues` fetch — this is the single most important mistake to avoid in all of TEAMWORK, because it is easy to write and looks correct until someone actually uses the Issues filter.
- Comparing `entry.issueId` directly against `project.id` — they are two different kinds of id that only look alike because both are plain numbers.

PART 2 — Solving the Homework

This part walks through the full, official Session 27 homework solution, step by step, in the exact order it was built: CORE first (HW-01 to HW-15), then STRETCH (HW-16 to HW-20, plus HW-22), then CHALLENGE (HW-21, a written-answer step with no code). Remember: all three tiers are required — CORE, STRETCH, and CHALLENGE are not optional add-ons, they only get progressively harder.

Step HW-01 — The `AppNotification` frontend type (CORE)

Goal: Give TypeScript a shape for a notification, matching exactly what the backend's notification endpoints already return.

Technical context: The same discipline LIVE/TEAMWORK already used for `Comment` / `ActivityEntry` — add the shared type before any component needs it.

File: `web/src/lib/types.ts`

Exact location: Add the new type right after the existing `ActivityEntry` type.

Before:

TSX

```tsx
export type ActivityEntry = {
  id: number;
  workspaceId: number;
  issueId: number | null;
  actorId: number | null;
  action: string;
  metadata: Record<string, unknown> | null;
  createdAt: string;
  actor: { id: number; name: string } | null;
};
```

Implementation:

TSX

```tsx
export type AppNotification = {
  id: number;
  userId: number;
  type: string;
```

```
  message: string;
  issueId: number | null;
  commentId: number | null;
  actorId: number | null;
  readAt: string | null;
  createdAt: string;
  actor: { id: number; name: string } | null;
};
```

After:

TSX

```
export type ActivityEntry = {
  id: number;
  workspaceId: number;
  issueId: number | null;
  actorId: number | null;
  action: string;
  metadata: Record<string, unknown> | null;
  createdAt: string;
  actor: { id: number; name: string } | null;
};

export type AppNotification = {
  id: number;
  userId: number;
  type: string;
  message: string;
  issueId: number | null;
  commentId: number | null;
  actorId: number | null;
  readAt: string | null;
  createdAt: string;
  actor: { id: number; name: string } | null;
};
```

Why this works: `message` is already a complete, human-readable string that the backend built once, at the moment the notification was created. The frontend never has to re-derive a sentence from `type` the way `ActivityFeed`'s own `describe()` does for activity — the backend already did that translation work for notifications.

In plain terms: this type just describes the shape of one notification row, the same way `Comment` and `ActivityEntry` describe their own rows. Nothing uses it yet.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. Nothing uses the new type yet.

Step HW-02 — `markNotificationReadAction` (CORE)

Goal: Create a Server Action that marks ONE notification as read — not called from anywhere yet.

Technical context: The backend's `PATCH /notifications/:id/read` endpoint already existed before this session. The homework gap is only the frontend's missing wiring to it.

File: `web/src/lib/actions/notifications.ts`

Exact location: A brand-new file, in the existing `lib/actions/` folder.

Before: File does not exist yet.

Implementation:

TSX

```tsx
"use server";

import { revalidatePath } from "next/cache";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";

export async function markNotificationReadAction(notificationId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/notifications/${notificationId}/read`, {
    method: "PATCH",
    token,
  });

  if (!res.ok) {
    throw new Error("Could not mark this notification read.");
  }

  revalidatePath(basePath);
}
```

After:

TSX

```tsx
"use server";

import { revalidatePath } from "next/cache";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";

export async function markNotificationReadAction(notificationId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/notifications/${notificationId}/read`, {
```

```
    method: "PATCH",
    token,
  });

  if (!res.ok) {
    throw new Error("Could not mark this notification read.");
  }

  revalidatePath(basePath);
}
```

Why this works — the important detail, `res.ok`. This is one of the most important patterns in this whole homework, so read it carefully.

This function is called directly from a button's `onClick`, not bound to a `<form>` — the same reason `startSprintAction` (Session 26) is a plain function rather than something shaped for `useActionState`. Marking one notification as read is not a form submission.

Here is the part students most often get wrong: `apiFetch` does NOT automatically throw an error for an ordinary failed request (a 404, a 403, a 500 — anything in the 4xx/5xx range). It only redirects automatically on a `401` (not logged in). For every other kind of failure, `apiFetch` just returns a normal response object, with `res.ok` set to `false`. Nothing throws on its own.

So WE have to check `res.ok` ourselves, and only call `revalidatePath()` after we've confirmed the request actually succeeded. Without the `if (!res.ok) { throw ... }` check, a failed `PATCH` request would silently fall through to `revalidatePath(basePath)` anyway — the page would refresh, and the UI would look like the notification was marked read, when really the backend rejected the request and nothing changed. The user would see no error at all, just a notification that still appears unread, which is confusing and silently wrong.

In plain terms: `apiFetch` is like a messenger that hands you back whatever the server said, good news or bad, without complaint. It's YOUR job to actually open the envelope (`res.ok`) and check whether it's good news before you tell the rest of the app "it worked."

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. Nothing calls this yet (HW-06).

Step HW-03 — `markAllNotificationsReadAction` (CORE)

Goal: Create a Server Action that marks EVERY notification as read — not called from anywhere yet.

Technical context: A pure, mechanical continuation of HW-02 — the identical shape, for the backend's `PATCH /notifications/read-all`.

File: `web/src/lib/actions/notifications.ts`

Exact location: A brand-new export, added at the end of the file, after `markNotificationReadAction`.

Before:

TSX

```
  revalidatePath(basePath);
}
```

Implementation:

TSX

```
  revalidatePath(basePath);
}

export async function markAllNotificationsReadAction(basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch("/notifications/read-all", {
    method: "PATCH",
    token,
  });

  if (!res.ok) {
    throw new Error("Could not mark all notifications read.");
  }

  revalidatePath(basePath);
}
```

After:

TSX

```
  revalidatePath(basePath);
}

export async function markAllNotificationsReadAction(basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch("/notifications/read-all", {
    method: "PATCH",
    token,
  });

  if (!res.ok) {
    throw new Error("Could not mark all notifications read.");
  }

  revalidatePath(basePath);
}
```

Why this works: The route is a plain string (`"/notifications/read-all"`), not a template literal — unlike `markNotificationReadAction` , this route needs no id interpolated into it, since it marks everything for the signed-in user. The same `res.ok` check from HW-02 applies here too: without it, a failed mark-all request

would be silently treated as a success.

In plain terms: this is HW-02's twin, just for "mark everything" instead of "mark one." Same pattern, same `res.ok` reasoning, nothing new to learn here.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. Nothing calls this yet (HW-07).

Step HW-04 — The Notifications page, basic skeleton (CORE)

Goal: Create a visitable page that fetches the signed-in user's notifications — no list rendered yet, just a count.

Technical context: The same page shape every other list page in this app already uses: fetch the data, guard with a token, render a placeholder for now.

File: `web/src/app/(app)/notifications/page.tsx`

Exact location: A brand-new file, in a new `app/(app)/notifications/` folder.

Before: File does not exist yet.

Implementation:

TSX

```
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { AppNotification } from "@/lib/types";

export default async function NotificationsPage() {
  const token = await requireSessionToken();

  const res = await apiFetch("/notifications/me", { token });
  const notifications: AppNotification[] = await res.json();

  return <main>{notifications.length} notifications</main>;
}
```

After:

TSX

```
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { AppNotification } from "@/lib/types";

export default async function NotificationsPage() {
  const token = await requireSessionToken();

  const res = await apiFetch("/notifications/me", { token });
```

```
  const notifications: AppNotification[] = await res.json();

  return <main>{notifications.length} notifications</main>;
}
```

Why this works: `requireSessionToken()` runs before the fetch — the same convention every protected page in this app already follows. An unauthenticated visitor never reaches the fetch at all.

In plain terms: this is the plainest possible version of the page — just enough to prove the fetch works and the route exists. The real list comes next.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. The real list comes next (HW-05).

Step HW-05 — The Notifications page, the real list (CORE)

Goal: The page now lists every notification, with unread ones visually highlighted — no mark-read buttons yet.

Technical context: A pure, mechanical transfer of the identical list shape `ActivityFeed` (TEAMWORK) already uses.

File: `web/src/app/(app)/notifications/page.tsx`

Exact location: The placeholder `<main>` grows real content.

Before:

TSX

```
  return <main>{notifications.length} notifications</main>;
```

Implementation:

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <h1 className="text-xl font-semibold">Notifications</h1>
      <ul className="flex flex-col gap-2">
        {notifications.map((n) => (
          <li key={n.id} className={`rounded-md p-3 text-sm ${n.readAt ? "bg-gray-50 dark:bg-
gray-900" : "bg-blue-50 dark:bg-blue-950"}`}>
            <p>{n.message}</p>
            <time className="text-xs text-gray-400">{new Date(n.createdAt).toLocaleString()}
</time>
          </li>
        ))}
        {notifications.length === 0 && <p className="text-sm text-gray-400">No notifications
yet.</p>}
```

```
      </ul>
    </main>
  );
```

After:

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <h1 className="text-xl font-semibold">Notifications</h1>
      <ul className="flex flex-col gap-2">
        {notifications.map((n) => (
          <li key={n.id} className={`rounded-md p-3 text-sm ${n.readAt ? "bg-gray-50 dark:bg-
gray-900" : "bg-blue-50 dark:bg-blue-950"}`}>
            <p>{n.message}</p>
            <time className="text-xs text-gray-400">{new Date(n.createdAt).toLocaleString()}
</time>
          </li>
        ))}
        {notifications.length === 0 && <p className="text-sm text-gray-400">No notifications
yet.</p>}
      </ul>
    </main>
  );
```

Why this works: `n.readAt ? ... : ...` is written directly inside the `className` template string. A read/unread status is already a boolean-shaped piece of data (`readAt` is either `null` or a real timestamp), so there is no need for a separate `isRead` variable — the ternary reads the data directly.

In plain terms: `readAt` tells you two things at once: whether it's read (is it `null`?) and, if so, exactly when. One field, two uses — that's why there's no separate "isRead" flag.

Verify / test:

BASH

```
npx tsc --noEmit && npm run build
```

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/notifications | grep -o "Notifications"
```

Expected result: `tsc` / `build` both exit clean; the notifications list appears.

Step HW-06 — `NotificationsList`, the basic skeleton (CORE)

Goal: Create a Client Component rendering the same list the page already renders — no mark-read buttons yet, and not used anywhere yet.

Technical context: A mark-read button needs `onClick`, which only a Client Component can offer — the same reason `DeleteIssueButton` (Session 26) is its own Client Component instead of inline Server Component JSX.

File: `web/src/app/(app)/notifications/NotificationsList.tsx`

Exact location: A brand-new file, a sibling of `page.tsx`.

Before: File does not exist yet.

Implementation:

TSX

```tsx
"use client";

import type { AppNotification } from "@/lib/types";

export function NotificationsList({
  notifications,
  basePath,
}: {
  notifications: AppNotification[];
  basePath: string;
}) {
  return (
    <ul className="flex flex-col gap-2">
      {notifications.map((n) => (
        <li key={n.id} className={`rounded-md p-3 text-sm ${n.readAt ? "bg-gray-50 dark:bg-gray-
900" : "bg-blue-50 dark:bg-blue-950"}`}>
          <p>{n.message}</p>
          <time className="text-xs text-gray-400">{new Date(n.createdAt).toLocaleString()}
</time>
        </li>
      ))}
      {notifications.length === 0 && <p className="text-sm text-gray-400">No notifications yet.
</p>}
    </ul>
  );
}
```

After:

TSX

```tsx
"use client";

import type { AppNotification } from "@/lib/types";

export function NotificationsList({
  notifications,
  basePath,
}: {
  notifications: AppNotification[];
  basePath: string;
}) {
  return (
```

```
    <ul className="flex flex-col gap-2">
      {notifications.map((n) => (
        <li key={n.id} className={`rounded-md p-3 text-sm ${n.readAt ? "bg-gray-50 dark:bg-gray-
900" : "bg-blue-50 dark:bg-blue-950"}`}>
          <p>{n.message}</p>
          <time className="text-xs text-gray-400">{new Date(n.createdAt).toLocaleString()}
</time>
        </li>
      ))}
      {notifications.length === 0 && <p className="text-sm text-gray-400">No notifications yet.
</p>}
    </ul>
  );
}
```

Why this works: `basePath` is accepted as a prop now, even though nothing in this step uses it yet. The mark-one/mark-all actions (HW-08/09) both need it to call `revalidatePath`. Declaring the full props shape upfront, before it's strictly needed, is the same convention `MentionTextarea`'s own `workspaceId` prop (LIVE) already used.

In plain terms: this component doesn't do anything new yet — it's the exact same list as HW-05, just moved into its own file so it CAN become interactive in the next steps.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. Nothing renders this component yet (HW-07).

Step HW-07 — Wire `NotificationsList` into the Notifications page (CORE)

Goal: The page now renders `NotificationsList` instead of its own inline JSX.

Technical context: Pure wiring — the component already exists (HW-06); this step renders it.

File: `web/src/app/(app)/notifications/page.tsx`

Exact location: One new import; the inline `<ul>` is replaced.

Before:

TSX

```
import type { AppNotification } from "@/lib/types";
```

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <h1 className="text-xl font-semibold">Notifications</h1>
      <ul className="flex flex-col gap-2">
        {notifications.map((n) => (
```

```
          <li key={n.id} className={`rounded-md p-3 text-sm ${n.readAt ? "bg-gray-50 dark:bg-
gray-900" : "bg-blue-50 dark:bg-blue-950"}`}>
            <p>{n.message}</p>
            <time className="text-xs text-gray-400">{new Date(n.createdAt).toLocaleString()}
</time>
          </li>
        ))}
        {notifications.length === 0 && <p className="text-sm text-gray-400">No notifications
yet.</p>}
      </ul>
    </main>
  );
```

Implementation:

TSX

```
import type { AppNotification } from "@/lib/types";
import { NotificationsList } from "./NotificationsList";
```

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <h1 className="text-xl font-semibold">Notifications</h1>
      <NotificationsList notifications={notifications} basePath="/notifications" />
    </main>
  );
```

After:

TSX

```
import type { AppNotification } from "@/lib/types";
import { NotificationsList } from "./NotificationsList";
```

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <h1 className="text-xl font-semibold">Notifications</h1>
      <NotificationsList notifications={notifications} basePath="/notifications" />
    </main>
  );
```

Why this works: `basePath="/notifications"` is a plain string literal, never a template literal — this page's own route never varies by any URL parameter.

In plain terms: nothing new is happening on screen here — the page looks identical. All that changed is WHERE the list-rendering code lives, so the next steps can add buttons to it.

Verify / test:

BASH

```
npx tsc --noEmit && npm run build
```

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/notifications | grep -o "Notifications"
```

Expected result: `tsc` / `build` both exit clean; the page renders identically to before — only its internal structure changed.

Step HW-08 — `NotificationsList`, the mark-one button (CORE)

Goal: Every unread notification now offers a working "Mark read" button.

Technical context: A mechanical transfer of the identical pending-state-plus-`onClick` shape `DeleteIssueButton` (Session 26) already uses, minus the confirm step — marking something read is never destructive, so no confirmation is needed.

File: `web/src/app/(app)/notifications/NotificationsList.tsx`

Exact location: One new import; a `markRead` function added before the `return`; one conditional button added inside each `<li>`.

Before:

TSX

```
import type { AppNotification } from "@/lib/types";
```

TSX

```
  return (
    <ul className="flex flex-col gap-2">
```

TSX

```
          <p>{n.message}</p>
          <time className="text-xs text-gray-400">{new Date(n.createdAt).toLocaleString()}
</time>
        </li>
```

Implementation:

TSX

```
import type { AppNotification } from "@/lib/types";
import { markNotificationReadAction } from "@/lib/actions/notifications";
```

TSX

```
  function markRead(notificationId: number) {
    markNotificationReadAction(notificationId, basePath);
  }
```

```
  return (
    <ul className="flex flex-col gap-2">
```

TSX

```
          <p>{n.message}</p>
          <time className="text-xs text-gray-400">{new Date(n.createdAt).toLocaleString()}
</time>
          {!n.readAt && (
            <button
              type="button"
              onClick={() => markRead(n.id)}
              className="mt-1 text-xs text-blue-600 hover:underline dark:text-blue-400"
            >
              Mark read
            </button>
          )}
        </li>
```

After:

TSX

```
import type { AppNotification } from "@/lib/types";
import { markNotificationReadAction } from "@/lib/actions/notifications";
```

TSX

```
  function markRead(notificationId: number) {
    markNotificationReadAction(notificationId, basePath);
  }

  return (
    <ul className="flex flex-col gap-2">
```

TSX

```
          <p>{n.message}</p>
          <time className="text-xs text-gray-400">{new Date(n.createdAt).toLocaleString()}
</time>
          {!n.readAt && (
            <button
              type="button"
              onClick={() => markRead(n.id)}
              className="mt-1 text-xs text-blue-600 hover:underline dark:text-blue-400"
            >
              Mark read
            </button>
          )}
        </li>
```

Why this works — fire-and-forget Server Actions. `markRead` is called from `onClick` but is NOT `await`ed. This is the same "fire-and-forget" pattern you've already used for other button-triggered Server Actions: the click fires the action, the action does its own backend request, and `revalidatePath` inside the action itself is what actually refreshes the page's data once the request finishes. There's no need to manage a loading state here by hand — this is a transfer of a pattern you already know, not a new concept.

In plain terms: clicking the button doesn't "wait" for anything on screen — it just tells the server action to go do its job, and when that job finishes, the page quietly updates itself because of `revalidatePath`.

Verify / test:

BASH
```
npx tsc --noEmit && npm run build
```

BASH
```
curl -s --cookie "session=$TOKEN" http://localhost:3001/notifications | grep -o "Mark read"
```

Expected result: `tsc` / `build` both exit clean; every unread notification shows a "Mark read" button.

Step HW-09 — `NotificationsList`, the mark-all button (CORE complete)

Goal: A "Mark all read" button now sits above the list.

Technical context: A mechanical continuation of HW-08 — the identical shape as `markRead`, for the other half of the notifications flow.

File: `web/src/app/(app)/notifications/NotificationsList.tsx`

Exact location: One new import; a `markAllRead` function added after `markRead`; one button added right before the returned `<ul>`.

Before:

TSX
```
import { markNotificationReadAction } from "@/lib/actions/notifications";
```

TSX
```
  function markRead(notificationId: number) {
    markNotificationReadAction(notificationId, basePath);
  }

  return (
    <ul className="flex flex-col gap-2">
```

TSX
```
    </ul>
  );
}
```

Implementation:

TSX

```tsx
import {
  markNotificationReadAction,
  markAllNotificationsReadAction,
} from "@/lib/actions/notifications";
```

TSX

```tsx
  function markRead(notificationId: number) {
    markNotificationReadAction(notificationId, basePath);
  }

  function markAllRead() {
    markAllNotificationsReadAction(basePath);
  }

  return (
    <>
      <button
        type="button"
        onClick={markAllRead}
        className="mb-2 text-xs text-blue-600 hover:underline dark:text-blue-400"
      >
        Mark all read
      </button>
      <ul className="flex flex-col gap-2">
```

TSX

```tsx
    </ul>
    </>
  );
}
```

After:

TSX

```tsx
import {
  markNotificationReadAction,
  markAllNotificationsReadAction,
} from "@/lib/actions/notifications";
```

TSX

```tsx
  function markRead(notificationId: number) {
    markNotificationReadAction(notificationId, basePath);
  }

  function markAllRead() {
    markAllNotificationsReadAction(basePath);
  }
```

```
  return (
    <>
      <button
        type="button"
        onClick={markAllRead}
        className="mb-2 text-xs text-blue-600 hover:underline dark:text-blue-400"
      >
        Mark all read
      </button>
      <ul className="flex flex-col gap-2">
```

TSX

```
    </ul>
    </>
  );
}
```

Why this works: A `<>` fragment is used, not a second `<div>`. The returned `<ul>` already carries its own layout, so wrapping both the button and the list in a plain fragment adds no extra DOM node for no reason — a Fragment (`<>...</>`) is an invisible wrapper that doesn't add anything to the actual HTML.

In plain terms: we needed to return TWO things (a button, then a list), but a component can only return one root element. A Fragment is just a way to group two things together without adding an extra, meaningless `<div>` around them.

Verify / test:

BASH

```
npx tsc --noEmit && npm run build
```

BASH

```
curl -s --cookie "session=$TOKEN" -X PATCH http://localhost:3000/notifications/read-all \
  -H "Authorization: Bearer $TOKEN" -o /dev/null -w "%{http_code}\n"
```

Expected result: `tsc` / `build` both exit clean; a `200` status code. CORE Notifications is now complete: a real list, mark-one, and mark-all, all working.

Step HW-10 — `deleteCommentAction` (CORE)

Goal: Create a Server Action that deletes a comment on the backend — not called from anywhere yet.

Technical context: The identical pattern `deleteIssueAction` (Session 26) already established. The backend's `DELETE /comments/:id` already existed before this session; the homework gap is only the frontend's missing wiring to it.

File: `web/src/lib/actions/comments.ts`

Exact location: A brand-new export, added at the end of the file, after `createCommentAction`.

Before:

TSX

```
  revalidatePath(basePath);
  return {};
}
```

Implementation:

TSX

```
  revalidatePath(basePath);
  return {};
}

export async function deleteCommentAction(commentId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/comments/${commentId}`, {
    method: "DELETE",
    token,
  });

  if (!res.ok) {
    throw new Error("Could not delete the comment.");
  }

  revalidatePath(basePath);
}
```

After:

TSX

```
  revalidatePath(basePath);
  return {};
}

export async function deleteCommentAction(commentId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/comments/${commentId}`, {
    method: "DELETE",
    token,
  });

  if (!res.ok) {
    throw new Error("Could not delete the comment.");
  }
```

```
  revalidatePath(basePath);
}
```

Why this works: This action throws a real `Error` instead of returning `{ error }`. That's different from `createCommentAction`'s own form submission, but it matches `deleteCommentAction`'s own calling style: it is going to be called directly from a button's `onClick` (HW-11/12), not from `useActionState`, which is exactly why `deleteIssueAction` (Session 26) also throws instead of returning an error object — there's no `useActionState` at the call site to catch a returned value.

In plain terms: this function doesn't care who calls it or how — it just does the delete request and either succeeds quietly or throws. What happens with that thrown error is the calling component's job (HW-12).

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. Nothing calls this yet (HW-11).

Step HW-11 — `DeleteCommentButton`, the basic skeleton (CORE)

Goal: Create a button, typed for one comment — not yet wired to delete anything.

Technical context: A mechanical transfer of the identical shape `DeleteIssueButton` (Session

1. already uses.

File:
`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/DeleteCommentButton.tsx`

Exact location: A brand-new file, a sibling of `DeleteIssueButton.tsx`.

Before: File does not exist yet.

Implementation:

TSX

```
"use client";

import { useState } from "react";

export function DeleteCommentButton({
  commentId,
  basePath,
}: {
  commentId: number;
  basePath: string;
}) {
  const [pending, setPending] = useState(false);

  return (
    <button
```

```
      type="button"
      disabled={pending}
      className="text-xs text-red-600 hover:underline disabled:opacity-50 dark:text-red-400"
    >
      Delete
    </button>
  );
}
```

After:

TSX

```
"use client";

import { useState } from "react";

export function DeleteCommentButton({
  commentId,
  basePath,
}: {
  commentId: number;
  basePath: string;
}) {
  const [pending, setPending] = useState(false);

  return (
    <button
      type="button"
      disabled={pending}
      className="text-xs text-red-600 hover:underline disabled:opacity-50 dark:text-red-400"
    >
      Delete
    </button>
  );
}
```

Why this works: `text-red-600` is the same color convention every other destructive button in this app already uses, so students (and reviewers) learn to recognize "red button = destructive action" on sight, consistently across the whole product.

In plain terms: right now this button just sits there and looks right — it doesn't delete anything yet. That comes next.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. The real `onClick` comes next (HW-12); nothing renders this component on a page yet (HW-13).

Step HW-12 — `DeleteCommentButton`, the confirm and delete (CORE)

Goal: Clicking the button actually deletes the comment, after a confirmation prompt.

Technical context: A mechanical continuation of HW-11 — the identical shape `DeleteIssueButton` (Session 26) already uses.

File:
`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/DeleteCommentButton.tsx`

Exact location: One new import; a `remove()` function added before the `return`; the button's `onClick` wired.

Before:

TSX
```tsx
import { useState } from "react";
```

TSX
```tsx
  return (
    <button
      type="button"
      disabled={pending}
      className="text-xs text-red-600 hover:underline disabled:opacity-50 dark:text-red-400"
    >
```

Implementation:

TSX
```tsx
import { useState } from "react";
import { deleteCommentAction } from "@/lib/actions/comments";
```

TSX
```tsx
  async function remove() {
    if (!confirm("Delete this comment? This cannot be undone.")) return;
    setPending(true);
    try {
      await deleteCommentAction(commentId, basePath);
    } finally {
      setPending(false);
    }
  }

  return (
    <button
      type="button"
      disabled={pending}
      onClick={remove}
      className="text-xs text-red-600 hover:underline disabled:opacity-50 dark:text-red-400"
    >
```

After:

TSX

```
import { useState } from "react";
import { deleteCommentAction } from "@/lib/actions/comments";
```

TSX

```
  async function remove() {
    if (!confirm("Delete this comment? This cannot be undone.")) return;
    setPending(true);
    try {
      await deleteCommentAction(commentId, basePath);
    } finally {
      setPending(false);
    }
  }

  return (
    <button
      type="button"
      disabled={pending}
      onClick={remove}
      className="text-xs text-red-600 hover:underline disabled:opacity-50 dark:text-red-400"
    >
```

Why this works: A plain `confirm()` dialog is used — the same reason `DeleteIssueButton` (Session 26) uses one: deleting a comment is a less reversible action, worth one extra, deliberate click before it actually happens. If the user cancels the browser dialog, `remove()` returns immediately and nothing else runs.

In plain terms: `confirm()` is a simple pop-up with OK/Cancel. If the user clicks Cancel, the function stops right there (`return`) and the delete request never even gets sent.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. Nothing renders this component on a page yet (HW-13).

Step HW-13 — Wire `DeleteCommentButton` into `CommentList` (CORE)

Goal: Every comment now offers a way to delete it.

Technical context — backend authorization, read carefully. The button renders for EVERY comment, with no client-side check for "can this user actually delete this one." That is deliberate. The backend's own `requireCanMutate` rule (the comment's author, or a workspace admin/owner) already enforces who this actually works for — this is the identical "button always renders, backend enforces independently" convention `DeleteIssueButton` (Session 26) already established, and it connects directly to that earlier pattern: the

frontend's job is only to DISPLAY the action; the backend remains the final authority on whether it is ALLOWED. If an unauthorized user somehow clicks Delete anyway, the backend rejects the request with a real `403`, and nothing is deleted.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/CommentList.tsx`

Exact location: One new import; `CommentList`'s props grow a `basePath`; the per-comment header grows a `justify-between` row and the button.

Before:

TSX
```
import { MentionText } from "@/components/ui/MentionText";
```

TSX
```
export function CommentList({ comments }: { comments: Comment[] }) {
```

TSX
```
        <li key={c.id} className="rounded-md bg-gray-50 p-2 text-sm dark:bg-gray-900">
          <p className="font-medium">{c.author?.name ?? "Deleted user"}</p>
          <p className="whitespace-pre-wrap">
```

Implementation:

TSX
```
import { MentionText } from "@/components/ui/MentionText";
import { DeleteCommentButton } from "./DeleteCommentButton";
```

TSX
```
export function CommentList({
  comments,
  basePath,
}: {
  comments: Comment[];
  basePath: string;
}) {
```

TSX
```
        <li key={c.id} className="rounded-md bg-gray-50 p-2 text-sm dark:bg-gray-900">
          <div className="flex items-center justify-between gap-2">
            <p className="font-medium">{c.author?.name ?? "Deleted user"}</p>
            <DeleteCommentButton commentId={c.id} basePath={basePath} />
          </div>
          <p className="whitespace-pre-wrap">
```

After:

TSX
```
import { MentionText } from "@/components/ui/MentionText";
import { DeleteCommentButton } from "./DeleteCommentButton";
```

TSX

```
export function CommentList({
  comments,
  basePath,
}: {
  comments: Comment[];
  basePath: string;
}) {
```

TSX

```
        <li key={c.id} className="rounded-md bg-gray-50 p-2 text-sm dark:bg-gray-900">
          <div className="flex items-center justify-between gap-2">
            <p className="font-medium">{c.author?.name ?? "Deleted user"}</p>
            <DeleteCommentButton commentId={c.id} basePath={basePath} />
          </div>
          <p className="whitespace-pre-wrap">
```

Why this works: `items-center justify-between` is the identical layout shape `MemberRow` /Issue Detail's own Reporter row (Session 25/26) already use: explanatory text on one side, an action on the other.

In plain terms: this step gives every comment a Delete button. Whether that click actually succeeds is decided by the backend, not by this component — the component's only job is to show the button and send the request.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: A real type error — `CommentList` 's own call site (HW-14) does not pass `basePath` yet. This is a deliberate, momentary broken state; HW-14 fixes it immediately next.

Step HW-14 — Pass `basePath` to `CommentList` (CORE Delete Comment complete)

Goal: The app compiles again — `CommentList` 's call site now passes the `basePath` that HW-13 made it require.

Technical context: Pure wiring — this restores the no-stop zone HW-13 deliberately opened (HW-13/14 are one continuous unit, not two independently stoppable steps).

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

Exact location: The `<CommentList>` call site grows one prop.

Before:

TSX

```
      <CommentList comments={comments} />
```

Implementation:

TSX

```
      <CommentList
        comments={comments}
        basePath={`/workspaces/${id}/projects/${projectId}/issues/${issueId}`}
      />
```

After:

TSX

```
      <CommentList
        comments={comments}
        basePath={`/workspaces/${id}/projects/${projectId}/issues/${issueId}`}
      />
```

Why this works: This is the identical `basePath` string `CommentForm` already receives — a comment deleted from this exact page must refresh this exact page.

In plain terms: HW-13 and HW-14 are really one single change split into two steps for teaching purposes. If you only did HW-13, the project would not compile. Always do both together.

Verify / test:

BASH

```
npx tsc --noEmit && npm run build
```

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/issues/1 | grep -o "Delete"
```

Expected result: `tsc` / `build` both exit clean; every comment shows a Delete button. CORE Delete Comment is now complete.

Step HW-15 — Link to the Notifications page (CORE complete)

Goal: A signed-in user can actually reach the Notifications page from anywhere in the app.

Technical context: Pure wiring — the last piece connecting HW-01..14 into one reachable product flow.

File: `web/src/components/layout/AccountBar.tsx`

Exact location: One new `<Link>`, added right before the signed-in user's own name link.

Before:

TSX

```
      <Link href="/account" className="text-blue-600 hover:underline dark:text-blue-400">
        {user.name}
      </Link>
```

Implementation:

TSX

```
      <Link href="/notifications" className="text-blue-600 hover:underline dark:text-blue-400">
        Notifications
      </Link>
      <Link href="/account" className="text-blue-600 hover:underline dark:text-blue-400">
        {user.name}
      </Link>
```

After:

TSX

```
      <Link href="/notifications" className="text-blue-600 hover:underline dark:text-blue-400">
        Notifications
      </Link>
      <Link href="/account" className="text-blue-600 hover:underline dark:text-blue-400">
        {user.name}
      </Link>
```

Why this works: Placed before the user's own name, on the one bar shown on every protected page — the same reach every other app-wide link in this bar already has.

In plain terms: without this link, the Notifications page would exist but be invisible — a student would have to type the URL by hand. This step makes it actually reachable.

Verify / test:

BASH

```
npx tsc --noEmit && npm run build
```

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces | grep -o "Notifications"
```

Expected result: `tsc` / `build` both exit clean; the link appears and navigates to the Notifications page.
HOMEWORK CORE is now fully complete: real notifications (list, mark-one, mark-all) and real comment deletion, both reachable end to end.

Step HW-16 — The Dashboard page, basic skeleton (STRETCH)

Goal: Create a visitable page that fetches the signed-in user's own assigned issues, across every workspace — no list rendered yet.

Technical context: The same page shape every other list page in this app already uses: fetch the data, guard with a token, render a placeholder for now.

File: `web/src/app/(app)/dashboard/page.tsx`

Exact location: A brand-new file, in a new `app/(app)/dashboard/` folder.

Before: File does not exist yet.

Implementation:

TSX

```tsx
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { Issue } from "@/lib/types";

type MyIssue = Issue & { project: { id: number; key: string; name: string; workspaceId: number }
};

export default async function DashboardPage() {
  const token = await requireSessionToken();

  const issuesRes = await apiFetch("/users/me/issues", { token });
  const issues: MyIssue[] = await issuesRes.json();

  return (
    <main>
      <h1>Dashboard</h1>
      {issues.length} assigned issues
    </main>
  );
}
```

After:

TSX

```tsx
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { Issue } from "@/lib/types";

type MyIssue = Issue & { project: { id: number; key: string; name: string; workspaceId: number }
};

export default async function DashboardPage() {
  const token = await requireSessionToken();

  const issuesRes = await apiFetch("/users/me/issues", { token });
  const issues: MyIssue[] = await issuesRes.json();

  return (
    <main>
      <h1>Dashboard</h1>
      {issues.length} assigned issues
    </main>
  );
}
```

Why this works: `MyIssue` is a LOCAL type (`Issue & { project: {...} }`), not an edit to the shared `Issue` type. `GET /users/me/issues` is the one route that nests a `project` object in its response — every other issue fetch in this app never does, so it would be wrong to permanently add that field to the shared `Issue` type just for this one page.

In plain terms: `MyIssue` means "a normal Issue, PLUS a bit of extra project info that only this one Dashboard endpoint happens to include." It's a local extension, not a change to the shared type everyone else uses.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. The real list comes next (HW-17).

Step HW-17 — The Dashboard page, the assigned-issue list (STRETCH)

Goal: The Dashboard page now lists every issue assigned to the signed-in user, each linking to its own Issue Detail page.

Technical context: A mechanical transfer of the identical card-list shape the Backlog page (Session 26) already uses.

File: `web/src/app/(app)/dashboard/page.tsx`

Exact location: Two new imports; the placeholder `<main>` grows real content.

Before:

TSX

```
import type { Issue } from "@/lib/types";
```

TSX

```
  return (
    <main>
      <h1>Dashboard</h1>
      {issues.length} assigned issues
    </main>
  );
```

Implementation:

TSX

```
import Link from "next/link";
import type { Issue } from "@/lib/types";
import { Card } from "@/components/ui/Card";
```

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <h1 className="text-xl font-semibold">Dashboard</h1>
```

```
      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Assigned to me ({issues.length})
      </h2>
      <div className="flex flex-col gap-3">
        {issues.map((issue) => (
          <Link
            key={issue.id}
            href=
{`/workspaces/${issue.project.workspaceId}/projects/${issue.project.id}/issues/${issue.id}`}
          >
            <Card>
              <p className="font-medium">{issue.key}: {issue.title}</p>
            </Card>
          </Link>
        ))}
        {issues.length === 0 && <p className="text-sm text-gray-400">Nothing assigned to you
right now.</p>}
      </div>
    </main>
  );
```

After:

TSX

```
import Link from "next/link";
import type { Issue } from "@/lib/types";
import { Card } from "@/components/ui/Card";
```

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <h1 className="text-xl font-semibold">Dashboard</h1>
      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Assigned to me ({issues.length})
      </h2>
      <div className="flex flex-col gap-3">
        {issues.map((issue) => (
          <Link
            key={issue.id}
            href=
{`/workspaces/${issue.project.workspaceId}/projects/${issue.project.id}/issues/${issue.id}`}
          >
            <Card>
              <p className="font-medium">{issue.key}: {issue.title}</p>
            </Card>
          </Link>
        ))}
        {issues.length === 0 && <p className="text-sm text-gray-400">Nothing assigned to you
```

```
right now.</p>}
      </div>
    </main>
  );
```

Why this works: `issue.project.workspaceId` / `issue.project.id` are read straight off the nested `project` object `MyIssue` carries. A cross-workspace page genuinely needs both ids to build the link, since neither one is implied by the current route (there's no `[workspaceId]` or `[projectId]` URL segment on `/dashboard` — it has to come from the data itself).

In plain terms: every other issue page in this app already "knows" its workspace/project from the URL. The Dashboard doesn't have that luxury — it shows issues from MANY workspaces at once — so it has to carry that information inside the issue data itself.

Verify / test:

BASH
```
npx tsc --noEmit && npm run build
```

BASH
```
curl -s --cookie "session=$TOKEN" http://localhost:3001/dashboard | grep -o "Assigned to me"
```

Expected result: `tsc` / `build` both exit clean; the assigned-issue list appears.

Step HW-18 — The Dashboard page, the activity fetch (STRETCH)

Goal: The page now also fetches the signed-in user's own recent activity, across every workspace — not rendered yet.

Technical context: A mechanical transfer of the identical `describe()`-style message shape `ActivityFeed` (TEAMWORK) already uses, now for a cross-workspace entry instead of a per-project one.

File: `web/src/app/(app)/dashboard/page.tsx`

Exact location: One new import; a local type and a `describeActivity` helper added before the component; the issues fetch grows an activity fetch.

Before:

TSX
```
import type { Issue } from "@/lib/types";
```

TSX
```
type MyIssue = Issue & { project: { id: number; key: string; name: string; workspaceId: number }
};
```

TSX
```
  const issuesRes = await apiFetch("/users/me/issues", { token });
  const issues: MyIssue[] = await issuesRes.json();
```

Implementation:

TSX

```tsx
import type { Issue, ActivityEntry } from "@/lib/types";
```

TSX

```tsx
type MyIssue = Issue & { project: { id: number; key: string; name: string; workspaceId: number }
};

type MyActivityEntry = ActivityEntry & {
  issue: { id: number; key: string; title: string } | null;
  workspace: { id: number; name: string; slug: string } | null;
};

function describeActivity(entry: MyActivityEntry) {
  const actor = entry.actor?.name ?? "Someone";
  const where = entry.issue?.key ?? entry.workspace?.name ?? "a workspace";
  return `${actor} -- ${entry.action} on ${where}`;
}
```

TSX

```tsx
  const issuesRes = await apiFetch("/users/me/issues", { token });
  const issues: MyIssue[] = await issuesRes.json();

  const activityRes = await apiFetch("/users/me/activity", { token });
  const activity: MyActivityEntry[] = await activityRes.json();
```

After:

TSX

```tsx
import type { Issue, ActivityEntry } from "@/lib/types";
```

TSX

```tsx
type MyIssue = Issue & { project: { id: number; key: string; name: string; workspaceId: number }
};

type MyActivityEntry = ActivityEntry & {
  issue: { id: number; key: string; title: string } | null;
  workspace: { id: number; name: string; slug: string } | null;
};

function describeActivity(entry: MyActivityEntry) {
  const actor = entry.actor?.name ?? "Someone";
  const where = entry.issue?.key ?? entry.workspace?.name ?? "a workspace";
  return `${actor} -- ${entry.action} on ${where}`;
}
```

TSX

```tsx
  const issuesRes = await apiFetch("/users/me/issues", { token });
  const issues: MyIssue[] = await issuesRes.json();
```

```
  const activityRes = await apiFetch("/users/me/activity", { token });
  const activity: MyActivityEntry[] = await activityRes.json();
```

Why this works — two independent queries, on purpose. Notice that this page makes TWO separate backend requests: `GET /users/me/issues` and `GET /users/me/activity`. They are kept as two independent calls instead of inventing one combined endpoint, because that is genuinely what the backend offers — there is no single route that returns both assigned issues and recent activity together. Also notice `entry.issue?.key ?? entry.workspace?.name` — `GET /users/me/activity`'s response nests BOTH `issue` and `workspace` on every entry, unlike the per-workspace activity route TEAMWORK used. An entry with no `issueId` at all (for example, a future member-management action) still has a workspace to name, so the fallback chain always finds something sensible to display.

In plain terms: "Assigned to me" and "Recent activity" are really two separate little features sharing one page. They don't need to talk to each other, so there's no reason to force them into one request — keeping them as two independent, already-existing queries is simpler and matches what the backend actually provides.

Verify:

BASH

```
npx tsc --noEmit
```

Expected result: Clean exit. `activity` / `describeActivity` are not read from anywhere yet (HW-19).

Step HW-19 — The Dashboard page, the activity render (STRETCH, Dashboard complete)

Goal: The Dashboard page now also shows a "Recent activity" section.

Technical context: Pure wiring — the data and the message helper already exist (HW-18); this step renders them.

File: `web/src/app/(app)/dashboard/page.tsx`

Exact location: The closing `</div>` of the assigned-issue list grows a sibling section.

Before:

TSX

```
        {issues.length === 0 && <p className="text-sm text-gray-400">Nothing assigned to you
right now.</p>}
      </div>
    </main>
```

Implementation:

TSX

```
        {issues.length === 0 && <p className="text-sm text-gray-400">Nothing assigned to you
right now.</p>}
      </div>
```

```
      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Recent activity
      </h2>
      <ul className="flex flex-col gap-2 text-sm">
        {activity.slice(0, 15).map((entry) => (
          <li key={entry.id} className="flex items-center justify-between gap-2 text-gray-500
dark:text-gray-400">
            <span>{describeActivity(entry)}</span>
            <time className="flex-shrink-0 text-xs text-gray-400">
              {new Date(entry.createdAt).toLocaleDateString()}
            </time>
          </li>
        ))}
        {activity.length === 0 && <p className="text-sm text-gray-400">No recent activity yet.
</p>}
      </ul>
    </main>
```

After:

TSX

```
        {issues.length === 0 && <p className="text-sm text-gray-400">Nothing assigned to you
right now.</p>}
      </div>

      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Recent activity
      </h2>
      <ul className="flex flex-col gap-2 text-sm">
        {activity.slice(0, 15).map((entry) => (
          <li key={entry.id} className="flex items-center justify-between gap-2 text-gray-500
dark:text-gray-400">
            <span>{describeActivity(entry)}</span>
            <time className="flex-shrink-0 text-xs text-gray-400">
              {new Date(entry.createdAt).toLocaleDateString()}
            </time>
          </li>
        ))}
        {activity.length === 0 && <p className="text-sm text-gray-400">No recent activity yet.
</p>}
      </ul>
    </main>
```

Why this works: `activity.slice(0, 15)` is used because the backend's own `GET /users/me/activity` already caps results at 50 rows — the Dashboard only ever needs a short, recent preview, not the whole list, so trimming it further to 15 keeps the page focused and readable.

In plain terms: the backend already limits how much activity it sends back; this just trims it down even further for a quick-glance Dashboard preview, rather than a full history page.

Verify / test:

BASH
```
npx tsc --noEmit && npm run build
```

BASH
```
curl -s --cookie "session=$TOKEN" http://localhost:3001/dashboard | grep -o "Recent activity"
```

Expected result: `tsc` / `build` both exit clean; the recent-activity section appears. The Dashboard page is now complete.

Step HW-20 — Link to the Dashboard page (STRETCH complete)

Goal: A signed-in user can actually reach the Dashboard page from anywhere in the app.

Technical context: Pure wiring — the last piece connecting HW-16..19 into one reachable product flow.

File: `web/src/components/layout/AccountBar.tsx`

Exact location: One new `<Link>`, added right before the "Notifications" link (HW-15).

Before:

TSX
```
      <Link href="/notifications" className="text-blue-600 hover:underline dark:text-blue-400">
        Notifications
      </Link>
```

Implementation:

TSX
```
      <Link href="/dashboard" className="text-blue-600 hover:underline dark:text-blue-400">
        Dashboard
      </Link>
      <Link href="/notifications" className="text-blue-600 hover:underline dark:text-blue-400">
        Notifications
      </Link>
```

After:

TSX
```
      <Link href="/dashboard" className="text-blue-600 hover:underline dark:text-blue-400">
        Dashboard
      </Link>
      <Link href="/notifications" className="text-blue-600 hover:underline dark:text-blue-400">
        Notifications
      </Link>
```

Why this works: Placed first, before "Notifications" and the user's own name — the Dashboard is this app's cross-workspace home, the first place a signed-in user's attention should be able to return to.

In plain terms: this is the same kind of link as HW-15, just for the Dashboard instead of Notifications — the "making a feature reachable" pattern is now a transfer, not something new to learn.

Verify / test:

BASH

```
npx tsc --noEmit && npm run build
```

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces | grep -o "Dashboard"
```

Expected result: `tsc` / `build` both exit clean; the link appears and navigates to the Dashboard page. **HOMEWORK STRETCH (Dashboard) is now complete:** the cross-workspace Dashboard, reachable end to end.

Step HW-21 — Why a `@mention` token stores the name, not only the id (CHALLENGE)

Goal: Explain, in your own words, why a comment's `@[Name](id)` token embeds the display name directly, instead of storing only the id (and resolving the name later) or only the name (with no stable reference at all).

Technical context: An independent-application challenge, combining the `@mention` mechanism LIVE already built with reasoning about WHY its storage format is shaped the way it is — no new product feature, no new code to write.

Official answer / model answer. A correct answer, in your own words, addresses both of the following points.

1. Why not only the id. If a comment stored only `@[4]` (an id, no name), `MentionText` would need to resolve every mentioned id to a name before it could render anything — a second network round-trip per comment, every single time it is shown, for data that never actually changes after the comment was posted. Storing the name directly means the comment is self-contained: it renders instantly from its own stored text, with no extra round trip.

2. Why not only the name. A display name is not a stable reference — two different users can share a name, and a user can rename their own account later. If the stored token were only `@Jane Doe`, the backend's own `extractMentionedUserIds` function (used to validate membership and to trigger notifications) would have no unambiguous way to know WHICH user was actually meant.

A complete answer connects both points to the same underlying idea: the token embeds BOTH pieces because each one answers a different question — the id is the stable reference the backend needs; the name is the display convenience the frontend needs — and neither one alone is sufficient for both jobs.

In plain terms: think of it like a work badge that shows both your employee ID and your name. The ID never changes and is what the security system actually checks (that's the backend's job). Your printed name is what a coworker reads at a glance without looking you up (that's the frontend's job). You need both, for two different readers.

One more thing worth knowing: what happens if a user's name changes later? Because the name is copied into the token text at the moment the comment is posted, an already-posted comment keeps showing the OLD name — it does not retroactively update. That is a reasonable, accepted trade-off for this app: comments are a

historical record of what was written and seen at the time, not a live, always-up-to-date view of a user's current profile.

Verify: There is no code to run for this step — it is graded on the written explanation covering both points above.

Expected result: A short written answer, in the student's own words, that clearly connects both reasons to the same underlying id-vs-name distinction.

Step HW-22 — A shared error boundary (STRETCH, HOMEWORK complete)

Goal: An uncaught error anywhere in the app (for example, an unauthorized `DeleteCommentButton` click, or a failed `markNotificationReadAction` / `markAllNotificationsReadAction` call with no local `try` / `catch`) now shows a friendly recovery screen instead of Next.js's own default, unstyled crash overlay.

Technical context. This app's established convention lets a Server Action throw (`deleteCommentAction`, `markNotificationReadAction`, `markAllNotificationsReadAction`) without wrapping every single call site in its own local `try` / `catch`. Several of this session's own call sites (`DeleteCommentButton`, `NotificationsList`'s fire-and-forget `markRead` / `markAllRead`) deliberately don't catch locally. A SHARED error boundary at the app's root is what keeps any one of those uncaught throws from ever reaching a raw, unstyled Next.js crash page.

File: `web/src/app/error.tsx`

Exact location: A brand-new file, at the app's own root — a sibling of the already-existing `web/src/app/not-found.tsx`.

Before: File does not exist yet.

Implementation:

TSX

```tsx
"use client";

import { useEffect } from "react";

export default function GlobalError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  useEffect(() => {
    console.error(error);
  }, [error]);

  return (
    <main>
      <h1>Something went wrong.</h1>
      <p>
```

```
        <button type="button" onClick={reset}>
          Try again
        </button>
      </p>
    </main>
  );
}
```

After:

TSX

```tsx
"use client";

import { useEffect } from "react";

export default function GlobalError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  useEffect(() => {
    console.error(error);
  }, [error]);

  return (
    <main>
      <h1>Something went wrong.</h1>
      <p>
        <button type="button" onClick={reset}>
          Try again
        </button>
      </p>
    </main>
  );
}
```

Why this works — explained simply.

- `"use client"` — a Next.js `error.tsx` file MUST be a Client Component. Why? Because it receives a real `reset()` function it calls to re-render the segment that threw, and that kind of interactive, resettable behavior only works in a Client Component — a Server Component has no way to hold or trigger that kind of live interaction.
- `error` — the actual error object that was thrown somewhere in the app. Next.js catches it and hands it to this component instead of letting it crash the whole page.
- `reset` — a function Next.js gives you that says "try rendering this part of the app again, as if the error never happened." Clicking the "Try again" button calls `reset()`, which asks Next.js to re-attempt rendering that part of the page.

- `useEffect(() => { console.error(error); }, [error]);` — this logs the technical error details to the browser console, for developers debugging the issue, WITHOUT ever showing the user the raw error message or stack trace. The user only ever sees the friendly "Something went wrong." message; the technical detail stays in the console where a developer can find it.

This page exists because not every Server Action call site has its own local `try` / `catch` — some intentionally throw and let this shared boundary catch them, so there's one single, friendly recovery screen for the whole app instead of needing to handle every possible failure one by one.

Verify / test:

BASH

```
npx tsc --noEmit && npm run build
```

Also verify by hand: temporarily make `DeleteCommentButton` call `deleteCommentAction` with a bogus comment id, click Delete, confirm the new error screen renders (never a raw Next.js crash overlay), and confirm "Try again" actually re-renders the page. Revert the temporary change afterward.

Expected result: `tsc` / `build` both exit clean; an uncaught error anywhere under the app now renders this shared screen. **HOMEWORK is now fully complete:** CORE (notifications, comment deletion), STRETCH (the Dashboard, the shared error boundary), and CHALLENGE (the `@mention` token written synthesis) — all required, and all working.

What the homework asked you to prove

CORE proved that a Notifications center and a comment-deletion flow are both mechanical transfers of `DeleteIssueButton`'s own already-taught shape — a fire-and-forget Server Action for a non-destructive action (marking read), a `confirm()`-gated one for a destructive action (deleting a comment), and a button that always renders while the backend enforces authorization independently. STRETCH proved the cross-workspace Dashboard assembles two genuinely independent, already-existing backend queries (`GET /users/me/issues`, `GET /users/me/activity`) into one page, reusing the identical card-list and activity-message patterns Backlog and `ActivityFeed` already established, plus a shared root error boundary that catches any uncaught throw from those same fire-and-forget/thrown-on-failure call sites. CHALLENGE proved you can explain, unprompted, why a `@mention` token's own stored shape embeds both the id and the display name, not just that it does.

Common mistakes to avoid (TEAMWORK + HOMEWORK)

- Building the Activity filter's issue-id set from the filtered `issues` list instead of a second, always-unfiltered fetch of this project's issues — then Activity entries for a real issue disappear the moment someone filters the Issues list to hide it. (TW-04)
- Comparing `entry.issueId` directly against a project's own `id` — they are two unrelated kinds of id that only look alike because both are plain numbers. (TW-04)
- Skipping the `res.ok` check in `markNotificationReadAction` / `markAllNotificationsReadAction` — `apiFetch` never throws on its own for an ordinary failed request, so without this check a failed request would be silently treated as a success. (HW-02/03)
- `await`-ing a fire-and-forget action inside `onClick` and blocking the UI on it, instead of trusting the action's own `revalidatePath` to refresh the page once it finishes. (HW-08/09)

- Giving `DeleteCommentButton` its own client-side check for "should this user see this button," instead of rendering it unconditionally and letting the backend's `403` do the real enforcing. (HW-11..14)
- Treating HW-13/14 as two independently stoppable steps, instead of one continuous no-stop zone — HW-13 alone leaves `CommentList` calling `DeleteCommentButton` without the `basePath` prop it now requires. (HW-13/14)
- Building one combined Dashboard fetch instead of the two real, independent calls this backend actually offers. (HW-18)
- Answering the `@mention` CHALLENGE with only ONE of the two required reasons, instead of connecting both to the same underlying idea. (HW-21)
- Omitting `"use client"` on `error.tsx`, or showing the user a raw error message instead of one generic, friendly message. (HW-22)

Exact testing instructions

BASH

```
cd api && PORT=3000 npm run start &
cd web && NEXT_PUBLIC_API_URL=http://localhost:3000 PORT=3001 npm run dev &

# Log in as the mentioned user
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login -H "Content-Type: application/json" \
  -d '{"email":"omar.admin@b4fboard.dev","password":"Password123!"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")

# TEAMWORK: per-project Activity feed
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1 | grep -o
"Activity"

# CORE: notifications list, mark-one, mark-all
curl -s http://localhost:3000/notifications/me -H "Authorization: Bearer $TOKEN" | python3 -m
json.tool
curl -s -X PATCH http://localhost:3000/notifications/7/read -H "Authorization: Bearer $TOKEN"
curl -s -X PATCH http://localhost:3000/notifications/read-all -H "Authorization: Bearer $TOKEN"

# CORE: delete a comment (as its own author)
curl -s -o /dev/null -w "%{http_code}\n" -X DELETE http://localhost:3000/comments/6 \
  -H "Authorization: Bearer $TOKEN"

# STRETCH: the Dashboard, both queries
curl -s --cookie "session=$TOKEN" http://localhost:3001/dashboard \
  | grep -o "Assigned to me\|Recent activity"
```

To deliberately break something and restore it: remove the `!n.readAt &&` guard around the "Mark read" button in `NotificationsList` — every notification, read or not, now shows the button. Restore the guard to fix it.

What you should understand after studying this

- Why Activity membership must be computed from a second, always-unfiltered fetch of a project's issues, never from the Issues UI's currently filtered list.
- Why a Server Action called directly from `onClick` (`markNotificationReadAction`, `deleteCommentAction`) needs no `useActionState`, and why only `deleteCommentAction` needs a `confirm()` first.
- Why `apiFetch` never throws automatically for an ordinary failed request, and why every action that matters must check `res.ok` itself before calling `revalidatePath`.
- Why `DeleteCommentButton` renders unconditionally, trusting the backend's own `requireCanMutate` check rather than hiding itself client-side.
- Why the Dashboard's two queries are independent calls, never one combined endpoint, since this backend offers no combined route.
- Why a shared root `error.tsx` boundary is needed when several of this session's own Server Actions deliberately throw with no local `try` / `catch` at their own call site.
- Why a `@mention` token's stored shape embeds both the id (the backend's stable reference) and the display name (the frontend's rendering convenience) — and why neither alone would be sufficient.